

Experience Engine: A Framework-Agnostic Protocol for Multi-Dimensional UI Experience Transitions

BASSEM HAZEM, Independent Researcher, Egypt

Multi-dimensional web interfaces may vary simultaneously along dimensions such as culture, theme, motion, typography, assets, resources, and component behavior. Existing technologies address many of these dimensions individually, but application code can still be responsible for coordinating cross-dimensional resolution, resource preparation, component adaptation, transition readiness, and state visibility. This paper presents the Experience Transition Protocol (ETP), a framework-agnostic runtime protocol implemented by Experience Engine. ETP models an experience as a resolved snapshot, computes a typed experience delta, derives the affected dependency closure, prepares heterogeneous dependent resources and component adaptations, and performs a guarded commit. We formalize the protocol and state invariants, implement a reference engine, and evaluate it against a strong manual Traditional+ baseline that independently reproduces the same tested safety semantics. Across generated and browser-based workloads, ETP preserves failure and stale-request invariants and coordinates heterogeneous consequences across culture, theme, and motion. Ablation experiments show that removing typed deltas, dependency closure, guarded commit, or prepare-before-commit introduces measurable extra work or safety failures under the corresponding workloads. The evaluation does not establish universal performance superiority, unique atomicity, or absolute novelty; locality results are limited to the tested synthetic dependency graphs. Exact React 18 E2E and Next.js SSR/RSC runtime validation were also performed, including request-time cookie personalization on a production Next.js setup, while third-party replication remains outside the present evaluation.

CCS Concepts: • **Human-centered computing** → **Interactive systems and tools**; *Web-based interaction*; • **Software and its engineering** → *Software architectures*.

Additional Key Words and Phrases: interactive systems, UI engineering, adaptive interfaces, runtime systems, design systems, transition protocols

ACM Reference Format:

Bassem Hazem. 2027. Experience Engine: A Framework-Agnostic Protocol for Multi-Dimensional UI Experience Transitions. In *Proceedings of EICS 2027*. ACM, New York, NY, USA, 6 pages.

1 Introduction

Modern UI systems increasingly adapt along multiple dimensions at runtime. A single user-visible change can affect language and direction, design tokens, component variants, component replacements, fonts, images, code, and motion. These concerns are often represented by separate mechanisms, leaving application code responsible for coordinating changes and deciding when the resulting state becomes visible.

We study a narrower systems problem: how to represent and execute a transition between multi-dimensional UI experiences as one reusable runtime operation without coupling the semantic ownership of culture, theme, and motion. We do not replace localization, design-system theming, animation libraries, or framework rendering. We define a protocol boundary at which these mechanisms can be coordinated.

The Experience Transition Protocol (ETP) makes an experience-level transition explicit, computes a typed difference, derives a dependency closure, prepares required consequences, and commits only an accepted result. The reference implementation, Experience Engine, provides framework adapters above a framework-agnostic core.

Author's Contact Information: Bassem Hazem, Independent Researcher, Egypt.

2027. Manuscript submitted to ACM

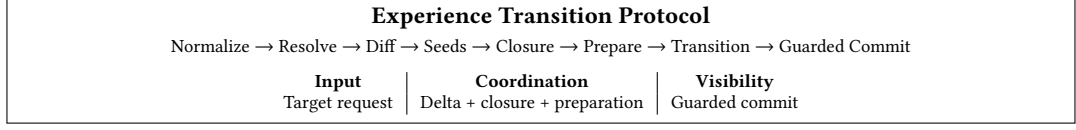


Fig. 1. ETP transition pipeline and the guarded visibility boundary.

The contribution is deliberately scoped. Prior work establishes substantial foundations for adaptive UI, model-based adaptation, multi-dimensional adaptation, dependency tracking, design-token resolution, and browser transition mechanisms. We do not claim to introduce those ideas individually. Our claim is that ETP provides a formalized and reusable protocol abstraction for coordinating them at the level of a multi-dimensional UI experience transition.

2 Problem Definition and Model

An experience is modeled as $E = (C, T, M, K, P, R)$, where C , T , and M denote culture, theme, and motion; K denotes resolved tokens and semantic properties; P denotes component decisions; and R denotes dependent resources.

Semantic ownership is explicit: culture owns locale, direction, formatting, and cultural presentation; theme owns visual tokens and theme-level component properties; motion remains independent. Fonts and assets may be context-dependent.

The protocol distinguishes token changes, variants, and replacements. The execution sequence is

Normalize → Resolve → Diff → Seeds → Closure → Prepare → Transition → Commit.

3 Protocol and Invariants

ETP defines a transition boundary around an immutable current snapshot and a proposed target snapshot. A typed delta identifies directly affected dimensions. Seed computation finds direct changes, dependency closure expands the affected work set, preparation establishes readiness without replacing the committed snapshot, and a guarded commit accepts only the latest valid prepared result.

The evaluated invariants cover deterministic resolution, immutable snapshots, failure preservation, stale dominance, atomic visibility, dependency closure, semantic ownership, and localized preparation under the tested dependency model. These invariants are not claimed to be unique to ETP; the Traditional+ baseline reproduces the same tested safety semantics.

4 Reference Implementation

Experience Engine separates a framework-agnostic core from React and Next.js adapters. The React-facing design uses an external-store integration and exposes provider and hook APIs. The inspected core source boundary contains no direct browser-global dependency.

Registries remain declarative and resolution results immutable. Preload acquires resources, whereas prepare establishes transition readiness. The public transition operation accepts a complete target request and coordinates normalization, resolution, delta computation, dependency closure, preparation, transition, and guarded commit.

Table 1. Final empirical evidence summary. Results are scoped to the reported workloads.

Evidence	Size	Outcome
Traditional+	10,000	0 partial commits; 0 stale overwrites
Locality DAG	10,000 nodes / 6 required	99.94% reduction vs full invalidation
Heterogeneous	5,000 random	0 semantic mismatches
Browser stress	301 cases	0 invariant violations
React 18 E2E	3 transitions	3/3 state matches; 3/3 React re-renders
Next dynamic SSR	2 cookie cases	2/2 exact server identities; HTTP 200
Ablation	10,000 transitions	Stage-specific failures/work effects
Reference model	5,000 random	0 failure-preservation violations

5 AI-Assisted Research Disclosure

Generative AI tools were used as research assistants during development of the artifact, including parts of research planning, architectural exploration, code drafting, test generation, experiment orchestration, data-analysis assistance, validation checks, and preparation of reproducibility materials. Human authors remain responsible for verifying the implementation, experimental results, references, claims, and final submission. The human author should verify that this disclosure exactly describes the actual workflow.

6 Independent Reference-Model Reproduction

A separate semantic reference implementation was written without importing Experience Engine. It independently implemented Normalize, Resolve, Diff, Seeds, Closure, Prepare, and Commit. The model passed 1,000 identity checks and 5,000 randomized transitions, including deterministic failure injection, with zero observed failure-preservation violations. This is an author-created independent implementation rather than third-party replication.

7 Evaluation

The evaluation combines formal/property-style testing, a separate reference-model reproduction, synthetic dependency workloads, heterogeneous adaptation workloads, browser stress tests, and a manual Traditional+ baseline.

7.1 Verification

Generated tests covered deterministic identity, immutable snapshots, direction handling, failure preservation, stale dominance, and dependency closure. No invariant violations were observed in the stored generated workloads.

7.2 Traditional+ Baseline

The strong manual baseline contains a central transaction controller, explicit dependency graph, semantic delta, prepare-before-commit, generation guards, and manual resource/component/motion coordination. Across 10,000 matched transitions with injected resource failures and stale-request scenarios, ETP and Traditional+ were semantically equivalent in the tested safety outcomes: no partial commits and no stale overwrites.

7.3 Locality

Structured dependency DAG experiments evaluated 1,000, 5,000, and 10,000-node graphs. In the strongest 10,000-node case with six required nodes, ETP prepared the six-node closure rather than invalidating all 10,000 nodes, a 99.94% reduction relative to full invalidation. These results characterize the tested synthetic dependency model.

7.4 Heterogeneous Consequences

Across 12 combinations of two cultures, two themes, and three motion modes, all 144 ordered transitions were exercised, followed by 5,000 random transitions. Consequences included tokens, variants, replacements, fonts, translations, assets, and motion. Semantic comparison against the strong manual baseline found 0 mismatches in the stored random workload.

7.5 Browser Stress

Real headless Chromium execution exercised 50 resource-failure cases, 50 stale-race cases, 101 rapid requests, and 100 mixed cases. The stored results showed zero invariant violations. In the rapid-request scenario, one request committed and 100 earlier requests were classified stale, with the final state matching the final request.

7.6 Framework Runtime Validation

The implementation was validated through an exact React 18.3.1 browser path and a Next.js 16.2.0 App Router path. In React, the test mounted `ExperienceProvider` with `createRoot`, used `useExperience` and `useSyncExternalStore`, executed three experience transitions through the engine, and verified that requested and final experience states matched in all three cases; an actual React re-render was observed for each transition. The observed switch latencies were 23.9, 34.0, and 27.4 ms, with CLS 0 and zero dropped frames in the stored samples. Long-task observation reported two entries per sample, so these measurements are treated as characterization rather than a performance claim.

For Next.js, the production build completed successfully using App Router. The initial server response was verified at HTTP 200 and contained the server-component markers for `'ar-EG'`, `'luxury'`, `'smooth'`, and RTL direction. A client-side benchmark then executed three transitions through the real React adapter and verified DOM convergence for each request. Finally, a dynamic route at `personalized` was exercised with request-time `experience` cookies. The same route returned distinct server-resolved identities: `'ar-EG::luxury::smooth'` for `experience=ar-EG.luxury.smooth` and `'en-US::light::instant'` for `experience=en-US.light.instant`, both with HTTP 200 and matching direction. This demonstrates request-time personalization in the tested Next.js production setup; it does not generalize to every deployment or cache topology.

7.7 Performance

Core microbenchmarks do not support a universal performance-superiority claim. In several measured scenarios, the engine introduced orchestration overhead. Performance is therefore treated as systems characterization.

7.8 Independent Reference-Model Reproduction

A separate semantic reference implementation was written without importing Experience Engine. It independently implemented Normalize, Resolve, Diff, Seeds, Closure, Prepare, and Commit. The model passed 1,000 identity checks

Table 2. Final claim boundary.

Claim	Status	Treatment
ETP protocol abstraction	Supported	Primary contribution
Typed delta + closure + preparation + commit	Supported	Core protocol
Synthetic locality	Scoped	Workload-specific
Browser stress	Scoped	Tested Chromium scenarios
Unique safety semantics	Rejected	Traditional+ reproduces tested semantics
Universal speedup	Rejected	Performance is characterization
First adaptive UI	Rejected	Prior-art audit does not support it
React 18 exact E2E	Supported	Direct runtime evidence
Next SSR/RSC	Supported	Direct runtime evidence within tested setup
Third-party replication	Pending	External replication still required

and 5,000 randomized transitions, including deterministic failure injection, with zero observed failure-preservation violations. This is an author-created independent implementation rather than third-party replication.

7.9 Final Evidence Audit

The final audit examined 11 project claims. The primary retained claims are the formalized ETP protocol abstraction and its coordination of typed delta, dependency closure, heterogeneous preparation, and guarded commit. Claims of unique safety semantics, universal performance superiority, and being the first adaptive or multi-dimensional UI framework were rejected. Locality and browser results remain explicitly scoped. React 18 E2E and Next.js SSR/RSC runtime validation are supported by direct runtime evidence within the tested setups; third-party replication remains pending. Arithmetic consistency checks passed.

8 Ablation

The ablation study used 500 generated dependency graphs and 10,000 transitions with 12% failure injection and 12% stale-request injection. Removing typed delta increased the prepared-work ratio to 1.358×. Removing dependency closure caused 7,661 downstream omissions. Removing the guard caused 1,050 stale overwrites. Removing prepare-before-commit caused 1,200 partial states.

9 Threats to Validity and Scope

The strongest tested safety semantics are reproducible by Traditional+. Locality is based on structured synthetic graphs. Browser evidence is scoped to controlled headless Chromium workloads. Exact React 18 E2E and Next.js SSR/RSC runtime validation were completed in the local test environments. The Next.js dynamic per-request test used request cookies on a production server and therefore does not establish behavior for every deployment, CDN, or cache topology. Artifact-level reproducibility and author-created independent reference modeling are not third-party replication. The work evaluates system properties rather than user outcomes.

The prior-art audit did not establish absolute novelty over adaptive UI, model-based adaptation, multi-dimensional adaptation, dependency tracking, or transactional UI techniques. The contribution claim is intentionally limited to the ETP abstraction and its explicit protocol framing.

10 Related Work

Adaptive and model-based UI research has long studied runtime transformation, context-aware adaptation, multi-dimensional adaptation, and mappings between abstract and concrete UI realizations [1, 3]. Culturally adaptive UI work studies the relationship between culture and interface presentation [2]. Design-token resolution and browser view transitions provide mature building blocks for visual and transition concerns [4, 5].

ETP is positioned as a coordination layer across these mechanisms. It does not replace localization, design systems, animation primitives, or browser transition APIs. Its scope is the experience-level protocol that makes the transition itself a first-class runtime operation, including typed change detection, dependency closure, heterogeneous preparation, and guarded acceptance.

11 Final Claim Boundary

The final evidence audit retains two primary claims: ETP is a formalized and reusable protocol abstraction for experience-level state transitions, and it coordinates typed delta, dependency closure, heterogeneous preparation, and guarded commit. Claims of unique safety semantics, universal performance superiority, and priority over the adaptive-UI literature are excluded. React 18 E2E and Next.js SSR/RSC runtime validation are supported by direct runtime evidence within the tested setups; third-party replication remains pending.

12 Conclusion

ETP is a formalized and reusable protocol abstraction for experience-level state transitions in multi-dimensional UI systems. The evidence supports the abstraction within the tested scope: formal properties pass, heterogeneous adaptation matches the strong manual baseline, browser stress tests preserve tested safety invariants, ablation exposes the role of protocol stages, and an independent reference model reproduces the core semantic behavior.

The evidence also constrains the claim: manual code can reproduce the tested safety semantics, the engine is not universally faster, synthetic locality results are workload-specific, and third-party replication remains open; the Next.js dynamic SSR evidence is scoped to the tested production configuration.

References

- [1] Erik G. Nilsson, Jacqueline Floch, Svein O. Hallsteinsen, and Erlend Stav. 2006. Model-based user interface adaptation. *Computers & Graphics* 30, 5 (2006), 692–701.
- [2] Katharina Reinecke and Abraham Bernstein. 2011. Improving Performance, Perceived Usability, and Aesthetics with Culturally Adaptive User Interfaces. *ACM TOCHI* 18, 2, Article 8.
- [3] Gaelle Calvary, Joelle Coutaz, Olivier Thevenin, and others. 2003. A unifying reference framework for multi-target user interfaces.
- [4] Design Tokens Community Group. 2025. Design Tokens Resolver Module 2025.10.
- [5] W3C CSS Working Group. CSS View Transitions Module Level 1.